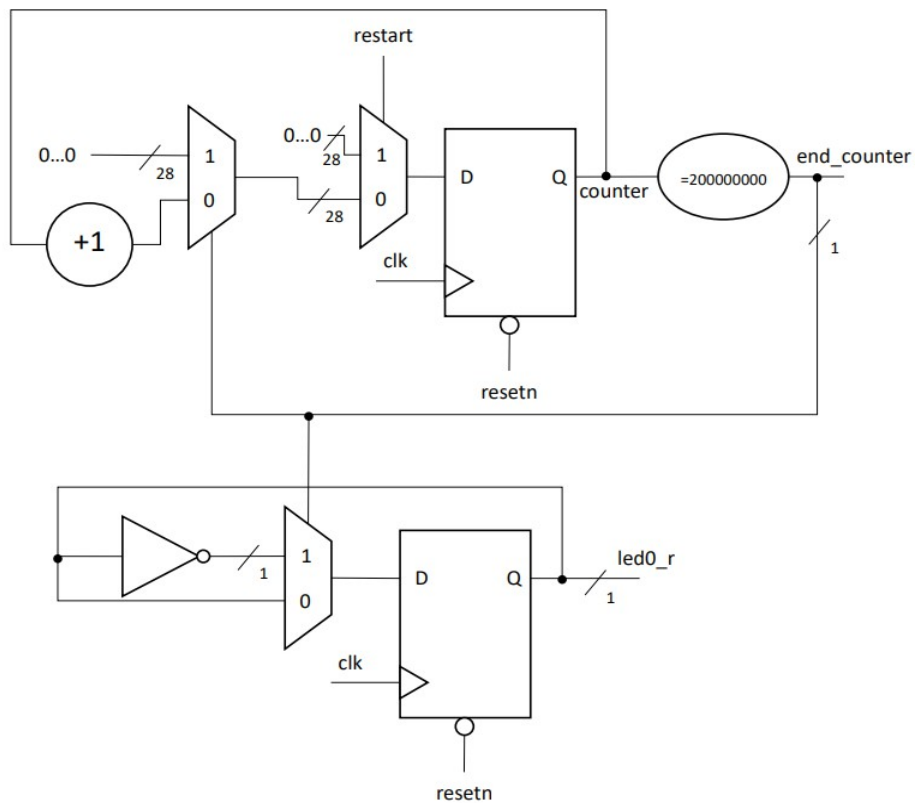


TP4 – Pilotage de LED et mémoire

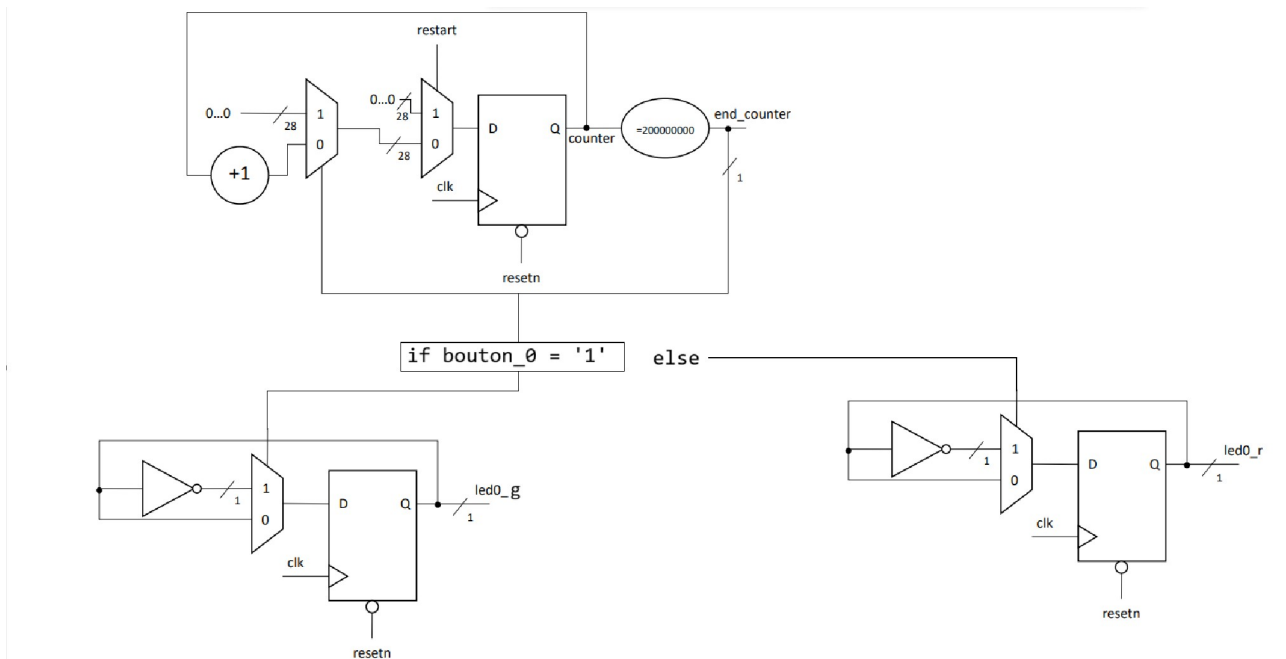
Partie 1

Questions

1. Créez une architecture RTL permettant de faire clignoter une LED (par exemple la led0_r) en utilisant le module Counter_unit du TP2 et une machine à état :



2. Modifiez votre architecture pour piloter une LED rouge et une LED verte. Lorsque le bouton_0 est appuyé, la LED verte est allumée, sinon la LED rouge est allumée.



3. Rédigez le code VHDL de votre architecture :

Fait

4. Réalisez une simulation en rédigeant un testbench. Que se passe-t-il si le bouton est pressé pendant plus d'un cycle d'horloge ?

La structure qu'on aurait envie d'adopter est la suivante :

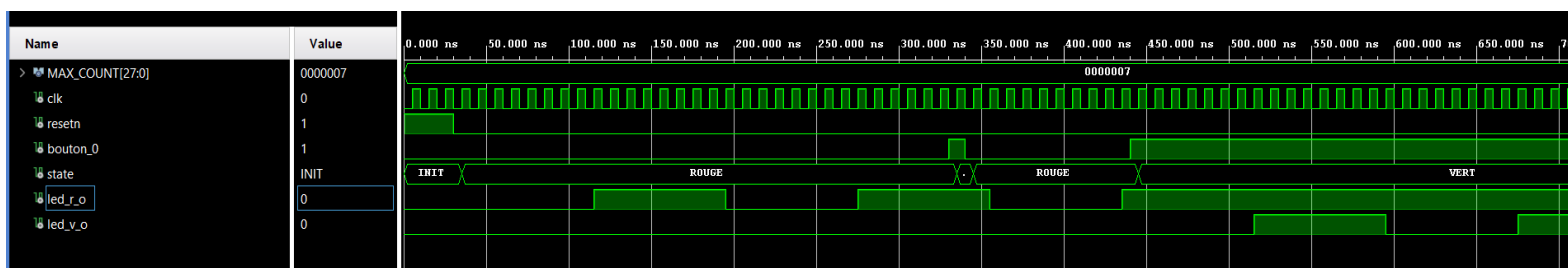
```
when ROUGE =>  
  if bouton = '1' then  
    etat <= VERT;  
  end if;
```

```
when VERT =>  
  if bouton = '1' then  
    etat <= ROUGE;  
  end if;
```

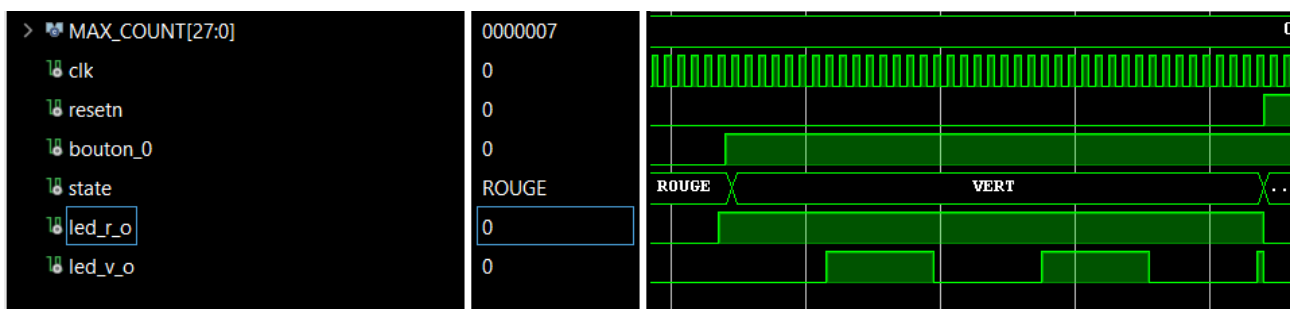
Mais il y a un problème : si on maintient le bouton appuyé pendant plusieurs cycles d'horloge, on risque de changer d'état à chaque coup d'horloge :

ROUGE → VERT → ROUGE → VERT → ROUGE... car bouton = '1' reste vrai

Je remarque un autre problème dans ma simulation:

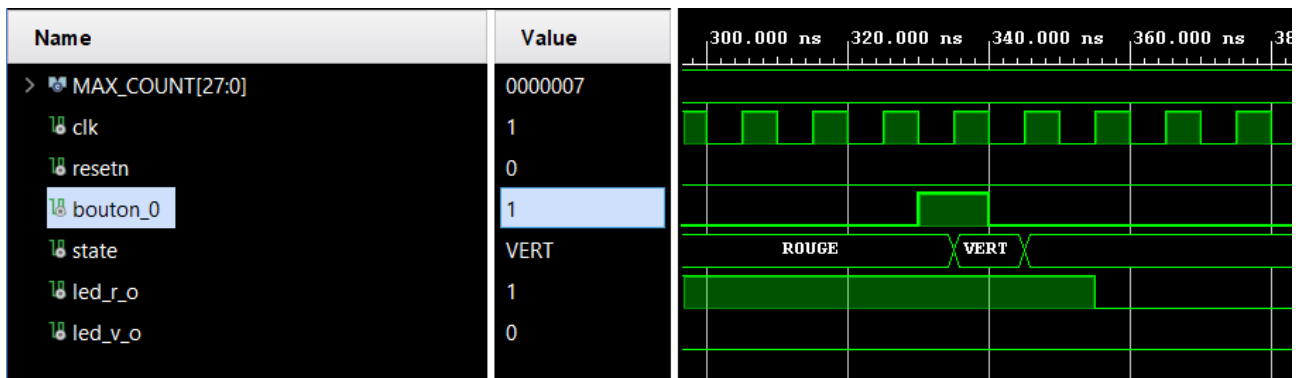


En effet, lorsque le bouton_0 est pressé pendant plus d'un cycle d'horloge, la LED verte (led_v_o) se met à clignoter aussi longtemps que le bouton_0 est pressé, on l'observe mieux ici :



On remarque que la LED verte (led_v_o) clignote aussi longtemps que le bouton_0 est à l'état haut, et cela pose problème !

En effet, cela veut dire que si l'on souhaite faire clignoter la LED verte pendant 2 secondes, il va falloir obligatoirement appuyer pendant 2 secondes sur le bouton_0 tout du long, ce qui n'est pas très pratique. On voit concrètement le problème ici :



Lorsque le bouton_0 est appuyé seulement pendant un cycle (10ns) la LED verte n'a pas le temps de se mettre à clignoter car le compteur n'a pas le temps de s'incrémenter jusqu'à la valeur max souhaité (199 999 999 coups d'horloge pour 2 secondes).

5. Que faudrait-il faire pour que la LED ne clignote en vert qu'une seule fois même si le bouton est maintenu ?

Il faut donc transformer bouton_0 = '1' en un événement, c'est à dire détecter un front montant de bouton_0.

Donc bouton_0 vient de passer de 0 à 1 => C'est un front montant.

On mémorise donc la valeur précédente du bouton : signal bouton_prec : std_logic := '0';

Puis, à chaque front d'horloge : bouton_prev <= bouton;

Et on détecte :

```
if bouton = '1' and bouton_prev = '0' then
```

```
-- nouvel appui
```

```
end if;
```

6. Ajoutez cette solution à votre architecture RTL.

Fait

7. Mettez à jour votre code VHDL et vérifiez votre résultat à la simulation.

```

bouton_0_prev <= bouton_0;

case state is
  when INIT =>
    led_r_s <= '0';
    led_v_s <= '0';
    state <= ROUGE;

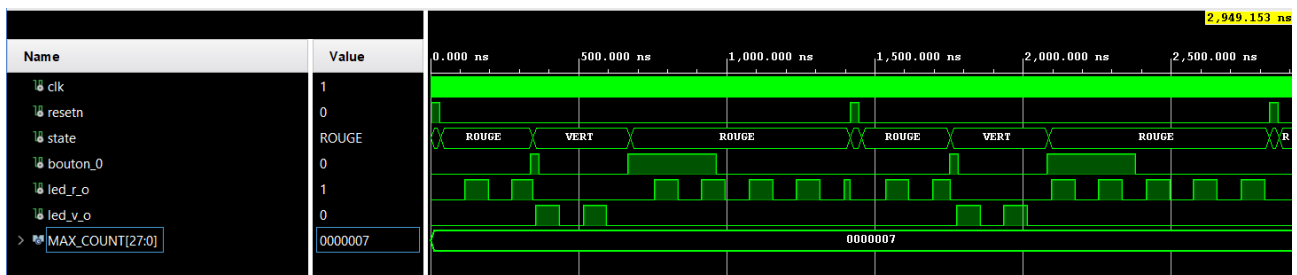
  when ROUGE =>
    if bouton_0 = '1' and bouton_0_prev = '0' then
      state <= VERT;
      led_r_s <= '0';
    else
      if end_counter_s = '1' then
        led_r_s <= not led_r_s;
      end if;
    end if;

  when VERT =>
    if end_counter_s = '1' then
      cycle_counter <= std_logic_vector(unsigned(cycle_counter) + 1);
    end if;

    if bouton_0 = '1' and bouton_0_prev = '0' then
      state <= ROUGE;
      led_v_s <= '0';
    else
      if end_counter_s = '1' then
        led_v_s <= not led_v_s;
      end if;
    end if;

  when others =>
    state <= INIT;
end case;

```



Lorsque reseth est à l'état '1' on se trouve bien dans l'état INIT, puis, lorsque reseth = '0' on rentre dans l'état ROUGE : la led_r_o (led rouge) clignote. Ensuite, lorsque le bouton_0 est pressé, on passe bien à l'état VERT : la led_v_o (led verte) clignote à son tour. Le bouton_0 est ensuite pressé et maintenu pendant plus longtemps et on repasse bien à l'état ROUGE et la led rouge clignote et ainsi de suite.

9. Ajouter la logique nécessaire pour piloter les entrées/sorties de votre module. - Le signal update doit recevoir 1 uniquement lorsque le bouton _0 vient d'être appuyé, maintenir le bouton enfoncé ne doit pas maintenir le signal update à 1 - Le signal color_code doit recevoir soit le code couleur « vert » si le bouton_1 est pressé, il doit recevoir le code couleur « bleu » sinon - Les LED R, G et B sont connectées avec les couleurs respectives de la LED_0

Fait

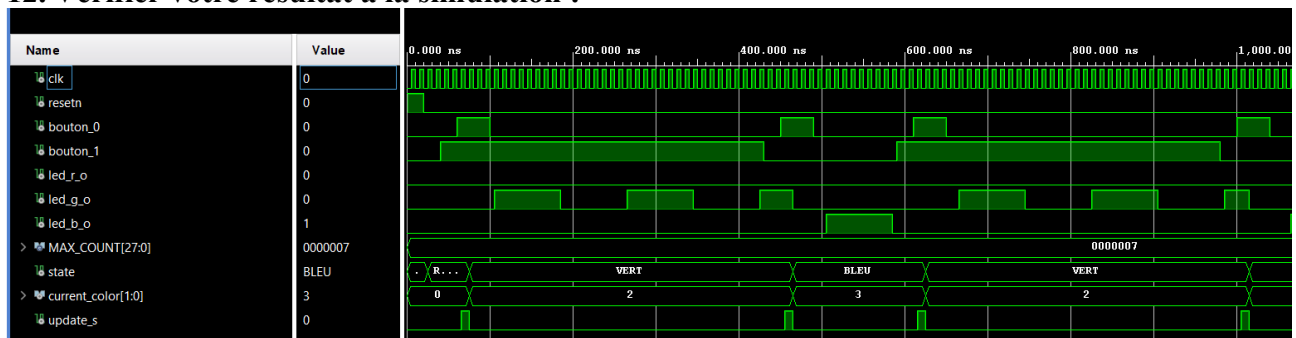
10. Ecrivez le code VHDL correspondant à votre architecture.

Fait

11. Ecrivez un testbench pour tester votre design.

Fait

12. Vérifier votre résultat à la simulation :



Lorsque resetn est à l'état haut, on se trouve dans l'état INIT. Lorsque le resetn est à l'état bas on rentre dans l'ETAT ROUGE, en attente qu'on appuie sur un bouton.

On remarque ensuite que, comme l'ont mentionné les spécifications, le bouton_1 est pressé (en attente du passage au VERT), le bouton 0 est ensuite appuyé ce qui fait effectivement passer notre machine état dans l'état VERT et notre led_g_o (led verte) clignote.

Dans l'autre cas, vers 450ns dans notre chronogramme, on remarque que le bouton_0 est appuyé SANS le bouton_1, on passe donc à l'état bleu.

La simulation valide bien le comportement souhaité.

13. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL :

```
-----
Detailed RTL Component Info :
+---Adders :
      2 Input    28 Bit      Adders := 1
+---Registers :
      28 Bit     Registers := 1
      1 Bit      Registers := 6
+---Muxes :
      2 Input    2 Bit      Muxes := 8
      4 Input    2 Bit      Muxes := 1
      2 Input    1 Bit      Muxes := 7
      4 Input    1 Bit      Muxes := 7
-----

Finished RTL Component Statistics
-----
```

Additionneurs :

1 additionneur **28 bits** (2 entrées) : correspond à la logique d'incrémement counter + 1 du compteur d'impulsions.

Registres :

- 1 registre de **28 bits** : stocke la valeur courante de counter.
- 6 registres de **1 bit** :
- 2 bits pour l'état de la FSM / machine à états (state_reg).
- Registres de mémorisation/synchronisation des entrées (bouton_0, bouton_1, update).
- Registres de basculement/pilotage des sorties (led_r, led_g, led_b).

Le synthétiseur a détecté une FSM 'state_reg' et lui a appliqué un encodage séquentiel sur 2 bits :

```
INFO: [Device 21-403] Loading part xc7z007sclg400-1
INFO: [Synth 8-802] inferred FSM for state register 'state_reg' in module 'led_driver'
-----
      State |           New Encoding |           Previous Encoding
-----
      init |           00 |           00
      rouge |          01 |           01
      bleu |          10 |           11
      vert |          11 |           10
-----
INFO: [Synth 8-3354] encoded FSM with state register 'state_reg' using encoding 'sequential' in module 'led_driver'
-----
```


14. Effectuez le placement routage et étudiez les rapports.

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 3,580 ns		Worst Hold Slack (WHS): 0,185 ns		Worst Pulse Width Slack (WPWS): 3,500 ns	
Total Negative Slack (TNS): 0,000 ns		Total Hold Slack (THS): 0,000 ns		Total Pulse Width Negative Slack (TPWS): 0,000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 62		Total Number of Endpoints: 62		Total Number of Endpoints: 37	
All user specified timing constraints are met.					

Notre WNS (chemin critique) a une marge de 3,580 ns ce qui est assez confortable, cohérent avec la simplicité de notre design.

« Toutes les contraintes de temps sont respectées »

15. Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

write_bitstream Complete 

Voici le lien de la démonstration :

<https://youtube.com/shorts/AbIEEW0cTPE?feature=share>

La démonstration valide bien notre design